

TorresBurgers

Guia completo para clonar, configurar, executar, entender e manter o projeto full stack.

Backend NestJS

PostgreSQL + TypeORM

Frontend React + Vite

Mobile Expo

Mercado Pago sandbox

Gerado em 22/09/2026

Workspace analisado: /home/crase/Área de trabalho/TorresBurguers

Objetivo: permitir que outro desenvolvedor suba o ambiente completo e entenda os fluxos principais de API, banco, web e mobile.

Sumário

1. Visão geral
2. Checklist de execução rápida
3. Estrutura do repositório
4. Pré-requisitos
5. Variáveis de ambiente
6. Banco de dados PostgreSQL
7. Backend NestJS
8. Frontend web
9. Frontend mobile
10. Autenticação, perfis e permissões
11. Mapa de endpoints da API
12. Modelagem do banco
13. Integrações externas
14. Fluxos principais
15. Uploads e imagens
16. Verificações executadas
17. Problemas conhecidos e recomendações
18. Troubleshooting

1. Visão geral

O TorresBurgers é um sistema de delivery com três aplicações principais: API backend em NestJS, painel/web de cliente e administração em React/Vite, e aplicativo mobile em React Native/Expo. A API centraliza autenticação, produtos, adicionais, pedidos, endereços, status da loja, pagamentos, promoções e upload de imagens.

Camada	Tecnologias	Responsabilidade
Backend	NestJS 11, TypeScript, TypeORM, PostgreSQL, JWT, Bcrypt, Sharp, Swagger	API REST, regras de negócio, autenticação, banco, uploads, pagamentos e integrações externas.
Web	React 19, Vite 7, TailwindCSS 4, Axios, Recharts, Lucide, Mercado Pago SDK	Cardápio do cliente, carrinho, checkout, perfil, pedidos, dashboard administrativo e gestão de cardápio/equipe.
Mobile	Expo 54, React Native 0.81, NativeWind, AsyncStorage, React Navigation, Axios	Experiência mobile de cliente e painel admin simplificado, com configuração de API via túnel Serveo.

Estado real do código: algumas configurações ainda estão hardcoded. O backend sempre escuta em 3000 ; o web usa `http://localhost:3000` diretamente em `frontend-web/src/services/api.ts` ; o mobile usa uma URL Serveo diretamente em `frontend-mobile/src/services/api.ts` , atualizada pelo script `npm run tunnel` .

2. Checklist de execução rápida

Este é o caminho curto para outro desenvolvedor subir tudo localmente.

1. Clonar o projeto

```
git clone <URL_DO_REPOSITORIO>
cd TorresBurguers
```

2. Preparar PostgreSQL

```
createdb -U postgres torresburgers
```

Se o usuário/senha forem diferentes, ajuste o arquivo `backend/.env` .

3. Configurar backend

```
cd backend
npm install
```

Crie `backend/.env` :

```
DB_HOST=localhost
DB_PORT=5432
DB_USERNAME=postgres
DB_PASSWORD=0000
DB_DATABASE=torresburgers

JWT_SECRET=troque_esta_chave_por_um_valor_grande_e_secreto
FOOTBALL_DATA_TOKEN=opcional_token_football_data
```

4. Subir a API

```
npm run start:dev
```

A API ficará em `http://localhost:3000` e o Swagger em `http://localhost:3000/api`.

5. Criar primeiro usuário admin

Em banco novo não há seed. Crie o admin pela rota pública de usuários:

```
curl -X POST http://localhost:3000/users \
-H "Content-Type: application/json" \
-d '{
  "name": "Admin Torres",
  "email": "admin@torresburgers.com",
  "phone": "21999999999",
  "password": "SenhaAdmin123!",
  "role": "ADMIN"
}'
```

6. Subir frontend web

```
cd ../frontend-web
npm install
npm run dev
```

Abra a URL exibida pelo Vite, normalmente `http://localhost:5173`.

7. Subir mobile

```
cd ../frontend-mobile
npm install
npm run tunnel
npm run start
```

Use `npm run tunnel` em uma janela de terminal para expor a API local via Serveo e atualizar `frontend-mobile/src/services/api.ts`. Depois rode o Expo em outra janela.

3. Estrutura do repositório

```
TorresBurguers/
├── backend/           API NestJS
```

```
├── src/auth/                login, JWT e roles
├── src/users/               usuários, perfis e avatares
├── src/products/           produtos e adicionais
├── src/orders/             pedidos, dashboard e logística
├── src/addresses/          endereços e busca de CEP
├── src/store/              status da loja aberta/fechada
├── src/payment/            PIX/cartão Mercado Pago sandbox
├── src/promotions/         banner futebol/promoções
├── src/common/             upload e manutenção de imagens
├── uploads/                imagens servidas em /uploads
├── frontend-web/           React + Vite
├──   └── src/
├──     ├── pages/           telas de cliente e admin
├──     ├── components/      modais, sidebar, cartão, footer
├──     ├── contexts/        auth, carrinho e tema
├──     ├── services/        Axios e serviços REST
├──     └── routes/           rotas web
├── frontend-mobile/        Expo + React Native
├──   ├── src/screens/       telas mobile
├──   ├── src/routes/        stacks de navegação
├──   ├── src/contexts/      auth, carrinho e tema
├──   ├── src/services/      Axios e serviços REST
├──   └── start-tunnel.js     automação de túnel Serveo
├── docs/                   documentação gerada
```

Não há `docker-compose.yml`, `Dockerfile` ou `.env.example` no workspace atual. O setup documentado aqui assume instalação local do PostgreSQL e execução por `npm`.

4. Pré-requisitos

Ferramenta	Versão recomendada	Observações
Node.js	20.19+ ou 22.12+	O Vite 7 avisa que exige Node 20.19+ ou 22.12+. No ambiente analisado, Node 18.19 ainda gerou build, mas a recomendação correta é Node 20 LTS.
npm	9+	O projeto usa <code>package-lock.json</code> em cada app.
PostgreSQL	14+ ou 16+	Testado no ambiente com PostgreSQL client 16.15.
Expo Go	Compatível com Expo 54	Necessário para testar o app no celular físico.
SSH	OpenSSH	Necessário para <code>frontend-mobile/npm run tunnel</code> usando Serveo.

5. Variáveis de ambiente

Backend

Crie `backend/.env` . O arquivo real é ignorado pelo Git.

Variável	Usada atualmente?	Descrição
<code>DB_HOST</code>	Sim	Host PostgreSQL. Default no código: <code>localhost</code> .
<code>DB_PORT</code>	Sim	Porta PostgreSQL. Default no código: <code>5432</code> .
<code>DB_USERNAME</code>	Sim	Usuário do banco. Default: <code>postgres</code> .
<code>DB_PASSWORD</code>	Sim	Senha do banco. Default: <code>0000</code> .
<code>DB_DATABASE</code>	Sim	Nome do banco. Default: <code>torresburgers</code> .
<code>JWT_SECRET</code>	Sim	Chave de assinatura JWT. Obrigatória para ambiente real.
<code>FOOTBALL_DATA_TOKEN</code>	Sim, opcional	Token da Football Data API para banner de jogos. Sem token, cai para promoção semanal local.
<code>PORT</code>	Não	Existe no <code>.env</code> , mas <code>main.ts</code> fixa <code>app.listen(3000, '0.0.0.0')</code> .
<code>JWT_EXPIRES_IN</code>	Não	Existe no <code>.env</code> , mas o módulo JWT fixa <code>expiresIn: '1d'</code> .
<code>MP_ACCESS_TOKEN</code>	Não	Existe no <code>.env</code> , mas o token do Mercado Pago está hardcoded no serviço de pagamento. Recomenda-se mover para esta variável.

Frontend web

O README cita `VITE_API_URL` , mas o código atual não usa essa variável. O arquivo `frontend-web/src/services/api.ts` possui `baseUrl: 'http://localhost:3000'` . Para trocar de API hoje, altere esse arquivo ou refatore para usar `import.meta.env.VITE_API_URL` .

Frontend mobile

O arquivo `frontend-mobile/src/services/api.ts` possui uma constante `API_URL` . O script `npm run tunnel` inicia `ssh -R 80:127.0.0.1:3000 serveo.net` , captura a URL pública e reescreve essa constante automaticamente.

6. Banco de dados PostgreSQL

Configuração TypeORM

O backend configura TypeORM em `backend/src/app.module.ts` com `synchronize: true`. Em desenvolvimento, isso cria e ajusta tabelas automaticamente a partir das entidades.

```
TypeOrmModule.forRoot({
  type: 'postgres',
  host: process.env.DB_HOST || 'localhost',
  port: parseInt(process.env.DB_PORT || '5432'),
  username: process.env.DB_USERNAME || 'postgres',
  password: process.env.DB_PASSWORD || '0000',
  database: process.env.DB_DATABASE || 'torresburgers',
  entities: [User, Product, Order, OrderItem, Address, Addon, StoreConfig],
  synchronize: true,
})
```

Importante: `synchronize: true` é conveniente em desenvolvimento, mas não deve ser usado em produção. Para produção, crie migrations TypeORM e desative o `synchronize`.

Criar banco limpo

```
createdb -U postgres torresburgers
```

Depois, inicie o backend. Ele criará as tabelas e a configuração inicial da loja em `store_config` com `id=1` e `is_open=true`.

Restaurar backup SQL

O IDE indicava um arquivo chamado `bdtorresburgers.backup.sql`, mas esse arquivo não estava presente no filesystem no momento em que este guia foi gerado. Se o backup for entregue junto do projeto, use:

```
createdb -U postgres torresburgers
psql -U postgres -d torresburgers -f bdtorresburgers.backup.sql
```

Se o dump tiver comandos de `owner/role` incompatíveis, edite o SQL ou restaure com um usuário compatível. Como o projeto usa `synchronize: true`, o backend pode completar colunas ausentes ao iniciar, mas isso não substitui migrations.

7. Backend NestJS

Comandos

Comando	Finalidade
<code>npm run start:dev</code>	Inicia a API em modo watch na porta <code>3000</code> .
<code>npm run build</code>	Compila TypeScript para <code>dist/</code> .
<code>npm run start:prod</code>	Executa <code>node dist/main</code> depois do build.
<code>npm test</code>	Roda testes unitários Jest do backend.

Comando	Finalidade
<code>npm run test:e2e</code>	Roda testes e2e, se configurados e atualizados.

URLs importantes

URL	Uso
<code>http://localhost:3000</code>	Base da API.
<code>http://localhost:3000/api</code>	Swagger/OpenAPI gerado pelo Nest.
<code>http://localhost:3000/uploads/<arquivo></code>	Arquivos estáticos da pasta <code>backend/uploads</code> .

Módulos principais

- `AuthModule` : login, JWT, estratégia Passport e roles.
- `UsersModule` : cadastro, atualização, soft delete, restauração e exclusão definitiva de usuários.
- `ProductsModule` : produtos, categorias, adicionais, soft delete e upload de imagem.
- `OrdersModule` : criação de pedidos, status, dashboard, logística e motoboys.
- `AddressesModule` : CRUD de endereços por usuário e proxy de CEP.
- `StoreModule` : status aberta/fechada da loja.
- `PaymentModule` : cartão, PIX, status e webhook simples.
- `PromotionsModule` : banner da home via Football Data API ou promoção semanal fallback.
- `CommonModule` : manutenção de imagens e otimização em massa.

8. Frontend web

Comandos

```
cd frontend-web
npm install
npm run dev
npm run build
npm run preview
```

Rotas web

Rota	Perfil	Descrição
<code>/</code>	Público/cliente	Home/cardápio.
<code>/cart</code>	Cliente	Carrinho e checkout.
<code>/signin</code>	Cliente	Login do cliente.
<code>/signup</code>	Cliente	Cadastro de cliente.
<code>/my-orders</code>	Cliente logado	Pedidos do cliente.

Rota	Perfil	Descrição
/profile	Cliente logado	Dados do perfil.
/login	Equipe	Login administrativo/funcionários.
/dashboard	ADMIN	Indicadores, gráficos e status da loja.
/orders	ADMIN/KITCHEN/COURIER/EMPLOYEE	Kanban/lista de pedidos e logística.
/products	ADMIN	Gestão de cardápio e lixeira de produtos.
/users	ADMIN	Gestão de equipe/usuários.
/trash	ADMIN	Lixeira de produtos e usuários.

Autenticação no web

O token é salvo em `localStorage` como `torresburgers.token` e o usuário como `torresburgers.user`.
O interceptor Axios injeta `Authorization: Bearer <token>`.

9. Frontend mobile

Comandos

```
cd frontend-mobile
npm install
npm run tunnel
npm run start
```

Outros comandos disponíveis: `npm run android`, `npm run ios` e `npm run web`.

Telas principais

O stack mobile contém `Home`, `ProductDetails`, `Cart`, `Checkout`, `AddAddress`, `Profile`, `SignUp`, `ForgotPassword`, `MyOrders`, `OrderDetails`, `PaymentPix`, `PaymentCard` e `AdminDashboard`.

Autenticação no mobile

O token é salvo no AsyncStorage como `@TorresBurgers:token` e o usuário como `@TorresBurgers:user`.
O interceptor Axios injeta o bearer token em cada requisição.

Uso do túnel

Celular físico normalmente não acessa `localhost` do computador. Por isso o script `start-tunnel.js` expõe a porta local `3000` via Serveo e troca a constante `API_URL` no arquivo mobile.

10. Autenticação, perfis e permissões

Roles

Role	Uso esperado
CLIENT	Cliente que faz pedidos e gerencia seus endereços.
ADMIN	Acesso total administrativo.
KITCHEN	Operação de pedidos/cozinha.
COURIER	Entregador/motoboy.
EMPLOYEE	Funcionário genérico com acesso parcial no web.

Login

POST /auth/login recebe email e password . Resposta:

```
{
  "access_token": "jwt...",
  "user": {
    "id": 1,
    "name": "Admin Torres",
    "email": "admin@torresburgers.com",
    "role": "ADMIN"
  }
}
```

Senha

A senha deve ter no mínimo 9 caracteres, pelo menos uma letra maiúscula e um número ou símbolo. As senhas são salvas com Bcrypt usando saltRounds = 10 .

Segurança de bootstrap: POST /users está público e aceita role . Isso facilita criar o primeiro admin, mas em produção deve ser protegido ou separado em um fluxo de seed controlado.

11. Mapa de endpoints da API

Base URL local: `http://localhost:3000` . Rotas protegidas usam `Authorization: Bearer <token>` .

Auth

Método	Rota	Acesso	Descrição
POST	<code>/auth/login</code>	Público	Autentica usuário por email e senha.

Users

Método	Rota	Acesso	Descrição
POST	<code>/users</code>	Público	Cria usuário. Aceita JSON ou multipart com campo <code>file</code> para avatar.
GET	<code>/users</code>	ADMIN	Lista usuários ativos.
GET	<code>/users/trash</code>	ADMIN	Lista usuários removidos por soft delete.
GET	<code>/users/:id</code>	JWT	Busca usuário por id.
PATCH	<code>/users/:id</code>	JWT	Atualiza usuário. Aceita avatar em multipart.
PATCH	<code>/users/:id/restore</code>	ADMIN	Restaura usuário da lixeira.
DELETE	<code>/users/:id</code>	JWT	Soft delete. Admin pode excluir qualquer um; usuário pode excluir a si mesmo.
DELETE	<code>/users/:id/permanent</code>	ADMIN	Exclusão definitiva.

Products

Método	Rota	Acesso	Descrição
GET	<code>/products</code>	Público	Lista produtos ativos com adicionais permitidos.
GET	<code>/products/:id</code>	Público	Busca um produto.
POST	<code>/products</code>	ADMIN	Cria produto. Usa multipart com <code>file</code> opcional.
PATCH	<code>/products/:id</code>	ADMIN	Atualiza produto. Usa multipart com <code>file</code> opcional.

Método	Rota	Acesso	Descrição
GET	/products/trash	ADMIN	Lista produtos em soft delete.
PATCH	/products/:id/restore	ADMIN	Restaura produto.
DELETE	/products/:id	ADMIN	Move produto para lixeira.
DELETE	/products/:id/permanent	ADMIN	Exclui produto definitivamente.

Addons

Método	Rota	Acesso	Descrição
GET	/addons	Público	Lista adicionais.
POST	/addons	Público no código atual	Cria adicional.
PATCH	/addons/:id	Público no código atual	Atualiza adicional.
DELETE	/addons/:id	Público no código atual	Soft delete de adicional.

Orders

Método	Rota	Acesso	Descrição
POST	/orders	CLIENT	Cria pedido. Bloqueia ADMIN e funcionários.
GET	/orders/my-orders	JWT	Lista pedidos do usuário logado.
GET	/orders	ADMIN/KITCHEN/COURIER	Lista todos os pedidos.
GET	/orders/:id	JWT	Busca pedido por id.
PATCH	/orders/:id	ADMIN/KITCHEN/COURIER	Atualiza status e outros campos parciais.
PATCH	/orders/:id/cancel	JWT	Cancela pedido. Cliente só cancela enquanto PENDING ; ADMIN pode cancelar.
GET	/orders/summary	ADMIN	Resumo do dashboard.
GET	/orders/charts	ADMIN	Dados de gráficos dos últimos 7 dias.
GET	/orders/drivers/list	ADMIN	Lista usuários com role COURIER .

Método	Rota	Acesso	Descrição
PATCH	/orders/:id/assign	ADMIN	Atribui motoboy e muda status para <code>DELIVERING</code> .
GET	/orders/:id/nearby	ADMIN	Sugere pedidos próximos no mesmo bairro.

Addresses

Método	Rota	Acesso	Descrição
GET	/addresses/cep/:cep	Público	Busca CEP via ViaCEP e fallback BrasilAPI.
POST	/addresses	JWT	Cria endereço para usuário logado.
GET	/addresses	JWT	Lista endereços do usuário logado.
GET	/addresses/:id	JWT	Busca endereço do próprio usuário.
PATCH	/addresses/:id	JWT	Atualiza endereço do próprio usuário.
DELETE	/addresses/:id	JWT	Remove endereço do próprio usuário.

Store, Payment, Promotions, Uploads e Maintenance

Método	Rota	Acesso	Descrição
GET	/store/status	Público	Retorna loja aberta/fechada e mensagens.
PATCH	/store/status	ADMIN	Atualiza status da loja.
POST	/payment/card	Público no código atual	Processa cartão via Mercado Pago sandbox.
POST	/payment/pix	Público no código atual	Gera pagamento PIX sandbox ou fallback visual.
GET	/payment/status/:id	Público no código atual	Consulta status do pagamento.
POST	/payment/webhook	Público no código atual	Webhook simples que apenas loga e responde <code>{ status: "ok" }</code> .
GET	/promotions/football	Público	Retorna banner de futebol ou promoção semanal.

Método	Rota	Acesso	Descrição
POST	/uploads	Público no código atual	Upload avulso de imagem, otimizada para WebP.
POST	/maintenance/optimize-images	ADMIN	Otimização em massa de imagens existentes.

12. Modelagem do banco

Enums importantes

Enum	Valores
UserRole	CLIENT , ADMIN , KITCHEN , COURIER , EMPLOYEE
ProductCategory	hamburgueres , entradas , acompanhamentos , bebidas , sobremesas , molhos , adicionais , combos
OrderStatus	PENDING , PREPARING , READY_FOR_PICKUP , DELIVERING , DONE , CANCELED
OrderType	DELIVERY , TAKEOUT
PaymentStatus	PENDING , PAID , FAILED

Tabelas e campos principais

Tabela	Campos principais	Relações
users	id , name , email único , password_hash , phone , avatar , role , timestamps , deleted_at	1:N com orders e addresses ; também pode ser driver em pedidos.
products	id , name , description , price , promotion_price , is_highlight , image , category , ingredients , available , deleted_at	N:N com addons via product_allowed_addons ; 1:N com order_items .
addons	id , name , description , price , image , available , category , deleted_at	N:N com produtos permitidos e itens de pedido.
orders	id , status , payment_status , type , total_price , delivery_fee , payment_method , change_for , payment_id , estimated_delivery_time , timestamps	N:1 com users , addresses e driver users ; 1:N com order_items .
order_items	id , quantity , observation , meat_point , removed_ingredients	N:1 com orders e products ; N:N com addons via order_item_addons .
addresses	id , zipCode , street , number , complement , neighborhood ,	N:1 com users .

Tabela	Campos principais	Relações
	city , state , nickname , latitude , longitude , created_at	
store_config	id , is_open , opening_message , closing_message	Configuração única, criada automaticamente com id=1 .

13. Integrações externas

Integração	Onde está	Necessário para funcionar?	Observações
Mercado Pago	backend/src/payment/ payment.service.ts , web PaymentForm , mobile PaymentCard	Sim para cartão/PIX real ou sandbox.	Access token e public key de teste estão hardcoded. Mover para .env antes de produção.
Football Data API	backend/src/promotions/promotions.service.ts	Opcional.	Usa FOOTBALL_DATA_TOKEN . Sem token, retorna promoção semanal local.
ViaCEP	backend/src/addresses/addresses.service.ts	Opcional, melhora cadastro.	Primeira tentativa de busca de CEP.
BrasilAPI CEP	backend/src/addresses/addresses.service.ts	Opcional, fallback.	Usada se ViaCEP falhar.
Serveo	frontend-mobile/start-tunnel.js	Necessário para celular físico fora do localhost.	Requer SSH e conexão externa.

14. Fluxos principais

Cadastro e login

1. Cliente ou admin cria usuário via POST /users .
2. Senha é validada pelo DTO e salva como password_hash com Bcrypt.
3. Login via POST /auth/login .
4. Frontend salva token e usuário em storage local.
5. Axios envia o token como bearer nas próximas requisições.

Pedido delivery

1. Cliente consulta /store/status e /products .

2. Cliente monta carrinho com produtos, adicionais, ponto da carne, observações e remoções de ingredientes.
3. Cliente escolhe ou cria endereço em `/addresses`.
4. Checkout pode passar por `/payment/pix` ou `/payment/card`.
5. Cliente cria pedido em `POST /orders` com role `CLIENT`.
6. Backend calcula frete por bairro e tempo estimado.
7. Equipe acompanha e muda status no painel.

Pedido retirada

Para `TAKEOUT`, `addressId` não é obrigatório, o frete é `0` e o tempo estimado é `15-20 min`.

Logística de entregas

1. ADMIN lista motoboys em `GET /orders/drivers/list`.
2. ADMIN atribui motoboy com `PATCH /orders/:id/assign`.
3. O pedido vai automaticamente para `DELIVERING`.
4. `GET /orders/:id/nearby` sugere pedidos `READY_FOR_PICKUP` do mesmo bairro e sem driver.

Pagamento

O Mercado Pago está em modo sandbox. O backend possui fallback e simulação: cartão rejeitado ou erro técnico pode retornar status aprovado simulado; PIX pode retornar um QR visual mock se o sandbox falhar. Isso é útil para desenvolvimento, mas deve ser removido/isolado em produção.

15. Uploads e imagens

Uploads de usuários, produtos e upload avulso passam pelo `OptimizeImagePipe`. O pipe valida JPG/JPEG/PNG/WebP, cria a pasta `uploads` se necessário, redimensiona para largura máxima de 800px e converte para WebP com qualidade 80.

O backend expõe a pasta `uploads` em `/uploads` via `ServeStaticModule`. Produtos salvam apenas o nome do arquivo; avatares de usuário normalmente salvam `/uploads/<arquivo>`.

```
POST /uploads
Content-Type: multipart/form-data
Campo do arquivo: file
Resposta: { "filename": "compressed-...webp" }
```

16. Verificações executadas

Comandos executados no workspace em 22/09/2026:

Comando	Resultado	Observação
<code>backend/npm run build</code>	OK	Compilação Nest concluída.
<code>backend/npm test -- --runInBand</code>	OK	1 suite, 1 teste passando.

Comando	Resultado	Observação
<code>frontend-web/npm run build</code>	OK	Build Vite gerado. Avisou que Node ideal é 20.19+ ou 22.12+.
<code>frontend-mobile/npx tsc --noEmit</code>	OK	TypeScript mobile sem erros.
<code>frontend-web/npm run lint</code>	Falha	Falha por regras estritas já existentes: <code>any</code> , <code>hooks</code> , <code>fast refresh</code> e regras novas do React.

17. Problemas conhecidos e recomendações

1. **Hardcoded secrets:** mover token Mercado Pago e public keys para variáveis de ambiente.
2. **Porta hardcoded:** usar `process.env.PORT || 3000` em `backend/src/main.ts`.
3. **Frontend web ignora `VITE_API_URL`:** refatorar `frontend-web/src/services/api.ts` para usar env.
4. **Mobile altera código ao rodar tunnel:** considerar usar arquivo de ambiente ou Expo Constants.
5. **`synchronize: true`:** trocar por migrations antes de produção.
6. **Rotas públicas sensíveis:** proteger `/addons`, `/uploads` e `/payment` conforme necessidade.
7. **Criação pública de admin:** após bootstrap, proteger criação de roles administrativas.
8. **CORS aberto:** hoje `origin: true`. Em produção, restringir domínios.
9. **Payment sandbox bypass:** não usar em produção.
10. **Sem Docker:** criar `docker-compose.yml` para PostgreSQL e, se desejado, API.
11. **Sem `.env.example`:** criar exemplos para backend, web e mobile.
12. **Lint web falhando:** decidir se as regras são desejadas ou ajustar/refatorar gradualmente.
13. **Backup SQL ausente no filesystem atual:** se o projeto depende de dados iniciais, versionar um seed seguro ou entregar dump sanitizado.

18. Troubleshooting

Backend não conecta no banco

- Confirme que PostgreSQL está rodando: `systemctl status postgresql`.
- Confirme que o banco existe: `psql -U postgres -l`.
- Confira `DB_HOST`, `DB_PORT`, `DB_USERNAME`, `DB_PASSWORD` e `DB_DATABASE`.

Web não acessa API

- Verifique se a API está em `http://localhost:3000`.
- Confira `frontend-web/src/services/api.ts`.
- Abra `http://localhost:3000/api` para validar que o backend está vivo.

Mobile não acessa API

- Rode o backend antes de abrir o túnel.

- Execute `npm run tunnel` em `frontend-mobile`.
- Confirme que `src/services/api.ts` recebeu a nova URL pública.
- Se Serveo cair, reinicie o túnel e o app Expo.

Pedido retorna "loja fechada"

- Consulte `GET /store/status`.
- Altere com ADMIN: `PATCH /store/status` enviando `{ "is_open": true }`.

Erro ao criar pedido delivery

- O usuário precisa ter role `CLIENT`.
- Pedidos `DELIVERY` exigem `addressId`.
- A loja precisa estar aberta.
- Produtos precisam existir no banco.

Upload falha

- Use campo multipart chamado `file`.
- Use JPG, JPEG, PNG ou WebP.
- Verifique permissão de escrita em `backend/uploads`.

Build web reclama do Node

Instale Node 20.19+ ou 22.12+ para alinhar com Vite 7.